INF201 : Projet 2023/2024
- Les dames chinoises
Version finale

Groupe:

Laghrouche Samir
Cranet Ael
Calderon Vargas Gabriel
Silva Morales Matias

```
type dimension = int;; (*restreint aux entiers strictement positifs*)

type case = int * int * int;; (*restreint au triplet tels (i,j,k) tels que i+j+k=0*)

type vecteur = int * int * int;; (*restreint au triplet tels (i,j,k) tels que i+j+k=0*)

type couleur = Vert | Jaune | Rouge | Noir | Bleu | Marron (*Les couleurs des joueurs*)
 | Libre
 | Code of string (*une chaine restreinte a 3 caracteres*);;

type case_coloree = case * couleur;;

type configuration = case_coloree list * couleur list * dimension;; (*sans case libre*)

type coup = Du of case * case | Sm of case list;;

let indice_valide (x:int) (dim:dimension) : bool =
 x >= -2*dim && x<= 2*dim;;

let est_case ((i,j,k):case):bool=
(i+j+k=0);;

let associe (a:'a) (l:('a*'b) list) (defaut:'b):'b = defaut;;
```

(*I- Première partie*)

(* Q1 *)

```
(*
1. i < -dim : i représente les lignes horizontales, avec une taille de 4dim+1 comprise entre [-
2dim, 2*dim]. Par conséquent, i peut être inférieur à -dim.
2. i > dim : De manière similaire à la précédente, i peut être supérieur à dim.
3. j < -dim : De la même manière, j peut être inférieur à -dim.
4. (i,j,k) = (2*dim, -dim, -dim) : Cette condition est vraie. Elle indique que i est égal à 2 fois la
dimension, j est égal à la dimension négative et k est égal à la dimension négative.
5. (i,j,k) = (-dim - 1, 1, dim) : Cette condition est également vraie. Elle indique que i est égal à
la
dimension négative moins 1, j est égal à 1 et k est égal à la dimension.
6. i >= -dim ^ j >= -dim ^ k >= -dim : Cette condition est fausse. Elle indique que i est un
entier
strictement positif, tout comme j et k, et que leur somme est égale à 0. Cependant, la somme
de -dim, -dim et -dim n'est pas égale à 0, donc cette condition est fausse. *)
```

(* Q2 *)

```
let est_dans_losange ((i, j, k) : case) (dim:dimension): bool = if i >= 2*dim || i <= -2*dim then
true
else false;;
```

(* Q3 *)

```
let est_dans_etoile (i, j, k : case) (dim:dimension) : bool =
 if ((i<= 2*dim && i>= -dim) && (j<= 2*dim && j>= -dim) && (k<= 2*dim && k>= -dim) ) || ((i>=
-
2*dim && i<= dim) && (j>= -2*dim && j<= dim )&& (k>= -2*dim && k<= dim ))
 then true
 else false ;;
```

(* Q4 *)

```
let rec tourner_case (m : int) (c : case) :case =
 let i,j,k = c in
 match m with
 | 0 -> (i,j,k) (* Si m est égal à zéro, la case reste inchangée *)
 | _ -> tourner_case (m-1) (-j,-k,-i) ;; (* Sinon, applique une rotation de la case (j, k, i) vers (-j,
-k, -i) *)
```

```
(* Q5 *)

let translate (i, j, k : case) (a, b, c : vecteur): case = (i + a, j + b, k + c);;
```

```
(* Q6 *)

let diff_case (i, j, k : case) (a, b, c : case): vecteur =
 (i - a, j - b, k - c);;
```

```
(* Q7 *)

let sont_cases_voisines (i, j, k : case) (a, b, c : case): bool =
 (* Cas où la première coordonnée est différente *)
 if (i, j + 1, k - 1) = (a, b, c) || (i, j - 1, k + 1) = (a, b, c) then true
 (* Cas où la deuxième coordonnée est différente *)
 else if (i + 1, j, k - 1) = (a, b, c) || (i - 1, j, k + 1) = (a, b, c) then true
 (* Cas où la troisième coordonnée est différente *)
 else if (i + 1, j - 1, k) = (a, b, c) || (i - 1, j + 1, k) = (a, b, c) then true
 else false;;
```

```
(* Q8 *)

type 'a option = Some of 'a | None ;;
(* La fonction calcul_pivot prend deux cases alignées en paramètres et retourne la case
pivot à michemin si elle existe *)
let calcul_pivot (i, j, k : case) (a, b, c : case) : case option =
 (* Vérification de l'alignement des cases *)
 if (i = a && j = -b && k = -c) || (i = -a && j = b && k = -c) || (i = -a && j = -b && k = c) then
 (* Vérification de l'existence de la case pivot *)
 let (v1, v2, v3) = ((i + a) / 2, (j + b) / 2, (k + c) / 2) in
 (* Vérification si les coordonnées de la case pivot sont paires *)
 if i = a then
 if v1 mod 2 = 0 then Some (v1, v2, v3)
 else if j = b then
 if v2 mod 2 = 0 then Some (v1, v2, v3)
 else if v3 mod 2 = 0 then Some (v1, v2, v3)
 else None
 else None
 else None
 else None;;
```

```
(* Q9 *)

let vec_et_dist ((i, j, k) : case) ((a, b, c) : case) : vecteur * int =
 (* Vérifie si les cases sont alignées *)
 if i = a || j = b || k = c then
 (* Calcul de la différence entre les coordonnées des deux cases *)
 let (i, j, k) = diff_case (i, j, k) (a, b, c) in
 (* Calcul de la distance en prenant le maximum des différences sur chaque axe *)
 let d = max (max i j) k in
 (* Calcul du vecteur de déplacement unitaire en divisant chaque coordonnée par la distance
*)
 ((i / d, j / d, k / d), d)
 else
 (* Si les cases ne sont pas alignées, retourne ((0, 0, 0), 0) *)
 ((0, 0, 0), 0);;
```

```
(*AFFICHAGE (fonctionne si les fonctions au dessus sont remplies)*)
(*transfo transforme des coordonnees cartesiennes (x,y) en coordonnees de case (i,j,k)*)
let transfo x y = (y, (x-y)/2,(-x-y)/2);;
let couleur2string (coul:couleur):string =
 match coul with
 | Libre -> " . "
 | Code s -> s
 | Vert -> " V "
 | Jaune -> " J "
 | Rouge -> " R "
 | Noir -> " N "
 | Bleu -> " B "
 | Marron -> " M ";;
let rec affiche_ligne (n:int) (m:int) (config:configuration) : string =
 let (lcc,_,dim)=config in
 if m = (4 * dim) + 1 then " " (*fin de ligne*)
 else
 let c = transfo m n in
 if not ((n+m) mod 2 = 0) || not (est_dans_etoile c dim) then (*ceci est une inter-case (case
inutile
d'un damier) ou hors de l'etoile*)
 " "^ affiche_ligne n (m + 1) config
 else (*ceci est une case ou bien en dehors du plateau*)
 (couleur2string (associe c lcc Libre)) ^ affiche_ligne n (m + 1) config;;
let affiche (config:configuration):unit =
 let (_,_,dim)=config in
 let rec affiche_aux n =
 if n = - 2 * dim - 1 then ()
 else
 begin
 print_endline (affiche_ligne n (-4*dim-1) config);
 print_endline "\n";
 affiche_aux (n - 1)
 end
 in
 affiche_aux (2*dim+1);;
let conf_1=([((0,0,0),Jaune)],[Jaune],2);;
affiche conf_1;;
let conf_reggae=([((0,-1,1),Vert);((0,0,0),Jaune);((0,1,-1),Rouge)],[Vert;Jaune;Rouge],1);;
affiche conf_reggae;;
let conf_vide=([],[],2);;
affiche conf_vide;;
(*A essayer apres avoir fait remplir_init
affiche (remplir_init [Code "Ali";Code "Bob";Code "Jim"] 3);;
*)
```

(*II- Deuxième partie*)

(* Q10v *)

```
let rec tourner_liste (l: 'a list): 'a list =
  match l with
  (* Cas où la liste est vide : retourne une liste vide *)
  | [] -> []
  (* Cas où la liste a un seul élément : retourne la liste inchangée *)
  | x::[] -> [x]
  (* Cas général où la liste a au moins deux éléments *)
  | t::q::h -> q::(tourner_liste (t::h))
;;

let rec der_liste (l:'a list): 'a =
  match l with
  (* Cas où la liste est vide : renvoie une erreur *)
  | [] -> failwith "Liste vide"
  (* Cas où la liste contient un seul élément : renvoie cet élément *)
  | [x] -> x
  (* Cas général : appelle récursivement der_liste sur le reste de la liste *)
  | _ :: q -> der_liste q
;;
```

(* Q11v *)

```
let rec remplir_segment (m:int) (c:case) : case list =
  let i,j,k = c in
  match m with
  (* Cas de base : lorsque m est égal à 1, retourne une liste contenant uniquement la
première case *)
  | 1 -> [(i,j,k)]
  (* Cas récursif : ajoute la case courante à la liste et appelle récursivement remplir_segment
avec m-1 *)
  | m -> (i,j,k) :: remplir_segment (m-1) (i,j+1,k-1)
;;
```

(* Q12v *)

```
let rec remplir_triangle_bas (m:int) (c:case) : case list =
  let i,j,k = c in
    match m with
    (* Cas de base : lorsque m est inférieur ou égal à 0, retourne une liste vide *)
    |m when m<=0 -> []
    (* Cas récursif : combine un segment horizontal et un triangle plus petit en dessous *)
    |_ -> remplir_segment m c @ remplir_triangle_bas (m-1) (i-1,j+1,k);;
```

```
(* Q13v *)

let rec remplir_triangle_haut (m:int) (c:case) : case list =
  let i,j,k = c in
  match m with
  (* Cas de base : lorsque m est inférieur ou égal à 0, retourne une liste vide *)
  |m when m<=0 -> []
  (* Cas récursif : combine un segment horizontal et un triangle plus petit au-dessus *)
  |_ -> remplir_segment m c @ remplir_triangle_haut (m-1) (i+1,j,k-1)
;;
```

```
(* Q14v *)

let rec colorie (c:couleur) (lc:case list): case_coloree list=
  match lc with
  (* Cas de base : liste vide, retourne une liste vide *)
  |[] -> []
  (* Cas récursif : applique la couleur à la première case et continue avec le reste de la liste
*)
  |pr::fin -> let (i,j,k) = pr in ((i,j,k),c)::(colorie c fin)
;;
```

```
(* Q15v *)

(*Fonction auxiliaire*)

let rec tourner_ma_liste (m:int) (l:case_coloree list): case_coloree list =
  match l with
  (* Cas de base : liste vide, retourne une liste vide *)
  | [] -> []
  (* Cas récursif : applique la fonction tourner_case à chaque case de la liste *)
  | (case, coul)::t ->
      (tourner_case (6-m) case, coul) :: tourner_ma_liste m t
let tourner_config (c:configuration):configuration =
  let (case_colore, couleur, dimension) = c in
  (* Rotation des cases *)
  let new_case = tourner_ma_liste (6 / (List.length couleur)) case_colore in
  (* Rotation des couleurs *)
  let new_couleur = tourner_liste couleur in
  (* Retourne la nouvelle configuration *)
  (new_case, new_couleur, dimension)
;;
```

```
type liste_joueur = couleur list
type case_option = None | Some of case
```

```
(* Assigne une couleur à chaque case d'une liste de cases *)
let rec colorie (coul: couleur) (lc: case list): case_coloree list =
  match lc with
  | [] -> []
  | [x] -> [(x, coul)]
  | hd::tl -> [(hd, coul)]@(colorie coul tl)


(* Calcule la longueur d'une liste *)
let rec length (l: 'a list): int =
  match l with
  | [] -> 0
  | _::tl -> 1 + length tl


(* Applique une fonction sur chaque case d'une liste de cases colorées *)
let rec fcaseColoree_list (f: int -> case -> case) (x: int) (l: case_coloree list): case_coloree list
=
  match l with
  | [] -> []
  | [(c, coul)] -> [((f x c), coul)]
  | hd::tl -> let (c, coul) = hd in [((f x c), coul)]@(fcaseColoree_list f x tl)


(* Remplit une liste de cases colorées pour chaque joueur *)
let rec remplir_List_Colorie (n: int) (l: couleur list) (cl: case list): case_coloree list =
  match l with
  | [] -> []
  | hd::tl -> fcaseColoree_list tourner_case n ((fcaseColoree_list tourner_case n (colorie hd
cl))@(remplir_List_Colorie n tl cl))


(*Q16v*)

(* Construit la configuration initiale à partir de la liste de joueurs et de la dimension *)
let rec remplir_init (lj: liste_joueur) (d: dimension): configuration =
  let long = length lj in
  let n = 6 / long in
  let c_init: case = (d + 1, -d, -(d + 1) - (-d)) in
  let l_case_init = remplir_triangle_bas n c_init in
  let ccList = remplir_List_Colorie d lj l_case_init in
  (ccList, lj, d)
;;


(* Q17v *)

(* La fonction associe recherche une clé dans une liste de couples clé-valeur et retourne la
valeur associée, ou une valeur par défaut si la clé n'est pas trouvée *)
let rec associe (a: 'a) (l: ('a * 'b) list) (defaut: 'b): 'b =
  match l with
  | [] -> defaut
  | (x, y)::fin -> if x = a then y else associe a fin defaut
```

```
(* La fonction quelle_couleur retourne la couleur associée à une case dans une
configuration, ou Libre si la case est libre *)
let quelle_couleur =
  fun (c: case) ->
    fun (conf: configuration) ->
      let (case_coul, _, _) = conf in
      associe c case_coul Libre
;;
```

```
(*Q18v*)

let supprime_dans_config (conf: configuration) (c: case): configuration =
  let lcase, lcoul, dim = conf in
  let rec aux (lcase: case_coloree list): case_coloree list =
    match lcase with
    | [] -> [] (* Si la liste est vide, on retourne une liste vide *)
    | (c', _)::fin when c' = c -> fin  (* Si la case correspond, on ne la rajoute pas dans la liste
résultat *)
    | pr::fin -> pr :: (aux fin) (* Sinon, on ajoute la case dans la liste résultat et on continue
avec le reste de la liste *)
  in
  (aux lcase), lcoul, dim (* On applique la fonction auxiliaire aux cases colorées de la
configuration et on retourne la configuration mise à jour *);;
```

```
(*Q19v*)

let est_coup_valide (conf: configuration) (cp: coup): bool =
  match cp with
  | Du(c1, c2) -> (* Si le coup est un déplacement unitaire *)
      let lcase, lcoul, dim = conf in
      let joueur_en_cours = List.hd lcoul in (* Récupération du joueur actuel *)
      (* Conditions de validité du coup *)
      (sont_cases_voisines c1 c2) && (* Vérifie si les cases c1 et c2 sont voisines *)
      ((associe c1 lcase Libre) = joueur_en_cours) && (* Vérifie si la case c1 contient un pion
du joueur actuel *)
      ((associe c2 lcase Libre) = Libre) && (* Vérifie si la case c2 est libre *)
      (est_dans_losange c2 dim) (* Vérifie si la case c2 est dans le losange Nord-Sud *)
  | Sm(_) -> failwith "Saut multiples non implementes"
;;
```

```
(*Q20v*)

let appliquer_coup (conf : configuration) (cp: coup) : configuration =
  match conf, cp with
  | _, Sm(_) -> failwith "Saut multiples non implementes"
  | (ccl, cl, dim), Du(c1, c2) -> (* Si le coup est un déplacement unitaire *)
      let (ccl', cl', dim') = supprime_dans_config (ccl, cl, dim) c1 in (* Supprime le pion de la
case de départ *)
      ((c2, List.hd cl')::ccl', cl', dim') (* Ajoute le pion dans la case d'arrivée *);;
```

```
(*Q21v*)

let mettre_a_jour_configuration (conf: configuration) (cp: coup) : configuration =
  match cp with
  | Du(c1, c2) -> (* Si le coup est un déplacement unitaire *)
      if est_coup_valide conf (Du(c1, c2)) then (* Vérifie si le coup est valide *)
        appliquer_coup conf (Du(c1, c2)) (* Applique le coup à la configuration *)
      else
        failwith "Ce coup n’est pas valide, le joueur doit rejouer"
  | _ -> failwith "error"
;;
```

```
(*Q22v*)

(* Vérifie si une case est libre dans une configuration *)
let case_libre (c: case) (conf: configuration) : bool =
  quelle_couleur c conf = Libre;; (* Utilise la fonction quelle_couleur pour vérifier si la case est
libre *)


(* La fonction est_libre_seg vérifie si toutes les cases entre c1 et c2 sont libres *)
let rec est_libre_seg = fun (c1: case) -> fun (c2: case) -> fun (conf: configuration) ->
  let (x1, y1, z1) = c1 in (* Coordonnées de la case de départ *)
  let (x2, y2, z2) = c2 in (* Coordonnées de la case d'arrivée *)
  let (v, u, w), dist = vec_et_dist c1 c2 in (* Vecteur et distance entre les deux cases *)
  match (x2, y2, z2), dist with
  | _, 1 -> true (* Si la distance est égale à 1, les deux cases sont voisines, donc la séquence
est libre *)
  | _, _ -> case_libre (x2+v, y2+u, z2+w) conf && est_libre_seg (x1, y1, z1) (x2+v, y2+u,
z2+w) (conf)   (* Vérifie si la case suivante est libre et si la séquence reste libre en continuant
avec la case suivante *) ;;
```

```
(*Q23v*)

let est_saut (x1, y1, z1 : case) (x2, y2, z2 : case) (l, li, dim : configuration) =
  let (u, v, w), d = vec_et_dist (x1, y1, z1) (x2, y2, z2) in  (* Calcul du vecteur et de la distance
entre les deux cases *)
  (d = 2) &&  (* Vérification que la distance entre les cases est de 2, indiquant un saut *)
  (est_dans_losange (x2, y2, z2) dim) &&  (* Vérification que la case d'arrivée est dans le
losange Nord-Sud *)
  (not (case_libre (x2 + u, y2 + v, z2 + w) (l, li, dim))) &&  (* Vérification que la case de saut
intermédiaire n'est pas libre *)
  (case_libre (x2, y2, z2) (l, li, dim)) &&  (* Vérification que la case d'arrivée est libre *)
  (est_dans_losange (x1, y1, z1) dim);;  (* Vérification que la case de départ est dans le
losange Nord-Sud *)
;;
```

```
(*Q24v*)

let rec est_saut_multiple (cl : case list) (conf : configuration) : bool =
  let res = est_saut (List.hd cl) (List.hd (List.tl cl)) conf in  (* Vérification du saut entre la
première et la deuxième case de la liste *)
  match res with
  | true -> est_saut_multiple (List.tl cl) conf  (* Si le saut est valide, on vérifie le saut entre les
cases suivantes *)
  | false -> false ;;  (* Si le saut n'est pas valide, on retourne faux *) ;;



(*Q25v*)

let est_coup_valide (conf:configuration) (cp:coup):bool =
  match cp with
  | Du(c1, c2) ->
      let lcase, lcoul, dim = conf in
      let joueur_en_cours = List.hd lcoul in
      (sont_cases_voisines c1 c2) &&                    (* Vérifie si les cases sont voisines *)
      ((associe c1 lcase Libre) = joueur_en_cours) &&    (* Vérifie si la case de départ est
occupée par le joueur en cours *)
      ((associe c2 lcase Libre) = Libre) &&             (* Vérifie si la case d'arrivée est libre *)
      (est_dans_losange c2 dim)                         (* Vérifie si la case d'arrivée est dans le
losange Nord-Sud *)
  | Sm([]) -> true                                  (* Un saut multiple vide est valide *)
  | Sm(pr::fin) -> est_saut_multiple (pr::fin) conf     (* Vérifie si le saut multiple est valide *)

let appliquer_coup (conf: configuration) (cp: coup) : configuration =
  let lcase, lcoul, dim = conf in
  match cp with
  | Du (c1, c2) ->
    let lcase4 = [(c1, Libre)] @ [(c2, quelle_couleur c1 conf)] in      (* Modifie les cases pour
refléter le déplacement *)
    let lcase2, lcoul2, dim2 = supprime_dans_config conf c1 in           (* Supprime la case de
départ *)
    let lcase3, lcoul3, dim3 = supprime_dans_config (lcase2, lcoul2, dim2) c2 in  (* Supprime
la case d'arrivée *)
    lcase4 @ lcase3, lcoul3, dim3
  | Sm (l) ->
    let lcase2 = [(List.hd l, Libre)] @ [(der_liste l, (quelle_couleur (List.hd l) conf))] in  (* Modifie
les cases pour refléter le saut multiple *)
    let lcase3, lcoul3, dim3 = supprime_dans_config (lcase, lcoul, dim) (List.hd l) in          (*
Supprime la première case du saut multiple *)
    let lcase4, lcoul4, dim4 = supprime_dans_config (lcase3, lcoul3, dim3) (der_liste l) in      (*
Supprime la dernière case du saut multiple *)
    lcase2 @ lcase4, lcoul4, dim4

let mettre_a_jour_configuration (conf:configuration) (cp:coup): configuration=
  if est_coup_valide conf cp then appliquer_coup conf cp       (* Vérifie si le coup est valide et
l'applique *)
  else failwith "Ce coup n'est pas valide, le joueur doit rejouer" ;;
```

(*III- Vérifier une partie*)

(*Q26v*)

```
(* La fonction score calcule le score du joueur protagoniste dans une configuration donnée.
   Le score d'un joueur est la somme des scores de ses pions.
   Le score d'un pion est sa coordonnée en x s'il appartient au joueur protagoniste.
   Sinon, son score est 0. *)

let score ((i,j,k) : configuration) : int =
  List.fold_left (fun acc ((x,y,z),coul) -> if (quelle_couleur (x,y,z) (i,j,k)) <> List.hd(j) then acc
else acc + x) 0 i ;;

(* La fonction score_gagnant calcule le score de la configuration gagnante pour le joueur
protagoniste.
   Dans une configuration gagnante, tous les pions du joueur protagoniste sont dans le camp
Nord.
   Le score gagnant est la somme des coordonnées x de tous les pions dans le triangle
supérieur du plateau. *)

let score_gagnant (dim:dimension) : int =
  List.fold_left (+) 0 (List.fold_left (fun acc (i,j,k) -> acc @ [i]) [] (remplir_triangle_haut dim
(dim+1,-dim,-1)));;
```

(*Q27v*)

```
(* La fonction gagne détermine si la configuration donnée est gagnante pour le joueur
protagoniste. *)

let gagne (conf: configuration) : bool =
  let (case, couleur, dim) = conf in (* Décomposition de la configuration en cases, couleurs et
dimension *)
  score conf = score_gagnant dim (* Vérification si le score de la configuration est égal au
score gagnant pour la dimension donnée du plateau *)
;;
```

```
(*Q28v*)

(* La fonction est_partie vérifie si une liste de coups correspond à une partie valide et
retourne la couleur du gagnant.
   Elle retourne Libre si aucun joueur n'a gagné.
   La configuration doit être tournée à chaque fois qu'un joueur joue.
   Les coups sont donnés toujours pour le joueur dont c'est le tour de jouer et qui se situe
alors le plus au Sud.
   Par exemple, si les deux premiers joueurs jouent le même coup, la liste de coups
commencera par deux fois le même coup. *)

let est_partie (cc_l, c_l, d: configuration) (cl: coup list) : couleur =
  if (cl = []) then
    Libre
  else
    let confi_final = (List.fold_left (fun x -> mettre_a_jour_configuration x) (cc_l, c_l, d) cl) in
    if gagne confi_final then
      let cc_lf, c_lf, df = confi_final in
      List.hd c_lf
    else
      Libre ;;
```

```
(*Q29v*)

(* Définition de la fonction triangle *)
let rec triangle (dim: dimension) ((i,j,k): case) : case list =
  match dim with
  | 1 -> [(i,j,k)] (* Cas de base : retourne la case actuelle *)
  | _ -> (remplir_segment dim (i,j,k)) @ triangle (dim-1) (i-1,j+1,k) (* Récursion : remplit le
segment et appelle récursivement la fonction *)


(* Définition de la fonction toute_cases *)
let toute_cases (config: configuration): case list =
  let lcase, lcouleur, dim = config in (* Décomposition de la configuration *)
  let lsortie = (triangle (2*dim) (-1,(-1*dim)+1,dim)) @ (remplir_segment (2*dim+1) (0,(-
1*dim),dim)) in (* Construction de la liste de sortie *)
  let rec aux (lcase: case list) = (* Fonction auxiliaire pour tourner les cases *)
    match lcase with
    | [] -> [] (* Cas de base : liste vide *)
    | case::fin -> (tourner_case (6/(List.length lcouleur)) case) :: (aux fin) (* Tourne la case et
appelle récursivement la fonction *)
  in
  (aux lsortie) @ (triangle (2*dim) (-1,(-1*dim)+1,dim)) (* Appelle la fonction auxiliaire et
ajoute le triangle *)
```

```
(* Définition de la fonction comparer_listes *)
let rec comparer_listes (elem: case) (lcase: case_coloree list): bool =
  match lcase with
  | [] -> false (* Cas de base : liste vide, retourne faux *)
  | (case, couleur)::t -> if elem = case then (* Vérifie si l'élément est égal à la case actuelle *)
                          true (* Retourne vrai si égal *)
                        else
                          comparer_listes elem t (* Appelle récursivement la fonction avec le reste
de la liste *)


(* Définition de la fonction enlever_case *)
let rec enlever_case (l_caselibre: case_coloree list) (l_caseoccupee: case_coloree list):
case_coloree list =
  match l_caselibre with
  | [] -> [] (* Cas de base : liste vide *)
  | (case, couleur)::t -> if comparer_listes case l_caseoccupee then (* Vérifie si la case est
occupée *)
                          enlever_case t l_caseoccupee (* Appelle récursivement la fonction avec le
reste de la liste *)
                        else
                          (case, couleur) :: (enlever_case t l_caseoccupee) (* Ajoute la case à la
liste résultante *)


(* Définition de la fonction liste_coups *)
let rec liste_coups (config: configuration) (case: case) (l_caselibre: case_coloree list):
(case*coup) list =
  match l_caselibre with
  | [] -> [] (* Cas de base : liste vide *)
  | (cs, couleur)::t -> if est_coup_valide (config) (Du(case,cs)) then (* Vérifie si le coup est
valide *)
                        (case, Du(case, cs)) :: (liste_coups config case t) (* Ajoute le coup à la liste
résultante et appelle récursivement la fonction *)
                      else
                        liste_coups config case t (* Appelle récursivement la fonction avec le reste
de la liste *)


(* Définition de la fonction coup_possibles *)
let coup_possibles (config: configuration) (case: case_coloree): (case*coup) list =
  let lcase, lcouleur, dim = config in (* Décomposition de la configuration *)
  let les_cases = colorie Libre (toute_cases config) in (* Colorie toutes les cases de la
configuration *)
  let cs, couleur = case in (* Décomposition de la case *)
  liste_coups (lcase, lcouleur, dim) cs (enlever_case les_cases lcase) (* Appelle la fonction
liste_coups avec les cases libres après enlèvement *)
```